# 实验指导：哈希长度扩展攻击与碰撞攻击

## 0. 实验概述

### 0.1 实验基本内容

实验任务：

1. 在不知道银行 API 密码的情况下，用长度扩展伪造一条管理员请求；
2. 用一对 MD5 碰撞块，做出两个 MD5 相同、行为不同的可执行程序。

第一件事利用的是 Merkle–Damgård 结构；第二件事利用的是 MD5 的碰撞抗性已经被攻破。 两件事都不需要自己造哈希，也不需要安装任何工具。

### 0.2 基本信息

| 编程量 | **25 处填空，约 30 行。**骨架已写好，只填关键位置 | | 交付物 | `data/forged_url.txt`、`data/good`、`data/evil`、两个 JSON + 实验报告 |

---

## 1. 原理速览

### 1.1 MD5 怎么处理一条消息

MD5 不是把整条消息一次算完，而是按 **64 字节一块**处理：

```
消息 → 补到 64 的倍数 → 逐块压缩 → 128 位摘要
```

补的规则就是填充：

```
0x80 | 若干 0x00 | 消息比特数（8 字节，小端序）
```

零的个数要让「0x80 + 零 + 8 字节长度」正好凑到 **56 mod 64。** 消息长度刚好是 55 或 56 字节时，填充会多出一整块，这是最常见的边界错误。

压缩函数每处理一块，都会更新一个 128 位的内部状态。处理完最后一块， 这个状态就是摘要。

### 1.2 长度扩展：校验身份只要摘要就够了么?

假设：

```
tag = MD5(secret || message)
```

你只知道 tag、message 和 len(secret)，不知道 secret 的值。

MD5 处理完 secret || message || padding 之后，内部状态正好等于 tag。 所以你可以把 tag 当作初始状态，继续压缩新的数据：

```
new_tag = MD5_from_state(tag, suffix)
        = MD5(secret || message || padding || suffix)
```

整个过程不需要 secret，只需要它的**长度**——因为长度决定了填充有多少字节。

这就是长度扩展攻击。它不要求找到碰撞，也不要求知道密钥。

## 1.3 为什么银行 API 会中招

本实验的 API 这样签名：

```
token = MD5(password || user=alice&command1=...&command2=...)
```

密码放在消息**前面**，正好是长度扩展能攻击的结构。

攻击者拿一条合法 URL，保留原 token，把消息改成：

```
原消息 || MD5 填充 || &command3=UnlockAllSafes
```

再用原 token 当初始状态，续算 &command3=...，得到新 token。 服务端重新计算 MD5(password || 新消息)，结果和攻击者给的新 token 一致， 于是返回 200 并执行 UnlockAllSafes。

服务端的密码没有被破解。出问题的是「把密码拼在消息前面再哈希」这个用法。

## 1.4 碰撞：两块不同的数据，同一个摘要

碰撞是指：

```
A ≠ B，但 H(A) = H(B)
```

MD5 的碰撞抗性在 2004 年被王小云等人攻破。本实验使用他们公布的一对 128 字节碰撞块：

- 两个文件都只有 128 字节；
- 只有 6 个字节不同；

- 不同的比特一共 6 个；
- MD5 完全相同；
- SHA-256 完全不同。

碰撞是「起始状态 + 消息」的性质：

- 在 A、B 前面加**相同前缀**，碰撞被破坏；
- 在 A、B 后面加**相同后缀**，碰撞保留。

原因还是 Merkle–Damgård：前缀不同，进入 A/B 时的起始状态就不同； 后缀相同，A/B 压缩完以后状态相同，后续计算自然完全一样。

## 1.5 碰撞能拿来做什么

如果两个文件的 MD5 相同，那么只校验 MD5 的软件会把它们当成同一个文件。

本实验把碰撞块放在文件最前面，后面接同一段 Python zipapp 代码：

```
good = 碰撞块 A || zipapp 代码
evil = 碰撞块 B || zipapp 代码
```

两段 zipapp 代码**逐字节相同**。程序读自己文件的第 `0x13` 字节， A 在这个位置是 `0x87`，B 是 `0x07`，于是打印不同的话。

结果是：

```
MD5(good) == MD5(evil)
SHA-256(good) != SHA-256(evil)
运行 good → I mean no harm.
运行 evil → You are doomed!
```

只检查 MD5 的下载站会把 evil 放行。

## 1.6 防御：HMAC 不是「把密钥拼进去再哈希」

正确的消息认证码是 HMAC：

```
HMAC(K, m) = H((K ⊕ opad) || H((K ⊕ ipad) || m))
```

它把密钥处理成两个块，分别参与内层和外层哈希。攻击者拿到 tag 以后， 没有办法把 tag 当成某个中间状态继续续写。

需要 MAC 时，用标准库 `hmac`，不要自己写 `H(secret || message)`。 比较 tag 时用 `hmac.compare_digest`，不要用 `==`。

---

# 2. 环境与数据

## 2.1 环境要求

只需要 Python 3.8 或更高版本，不需要安装任何第三方库。

```
python --version
```

先跑环境自检：

```
cd hash-lab1/code
python check_env.py
```

它会检查解释器版本、标准库、数据文件、MD5 参照实现、填空机制， 最后打印填空进度。全绿时收尾句是「全部通过 —— 可以开始实验了。」

> ⚠ 不要用 `python -I` 或 `python -E` 运行本包的脚本。 `-I` 会把脚本目录移出 `sys.path`，`from fill import ...` 会失败。

## 2.2 目录结构

```
hash-lab1/
├── code/                       ← 你要改的代码
│   ├── fill.py                 填空机制，不要修改
│   ├── check_env.py            环境自检（课前作业）
│   ├── md5_soft.py             纯 Python MD5，支持从中间状态续算
│   ├── api_common.py           银行服务端（签发 + 校验）
│   ├── issue_url.py            签发样例 URL
│   ├── task1_md5.py            【6 空】填充与续写
│   ├── task2_extend.py         【8 空】长度扩展攻击【主交付物】
│   ├── task3_collision.py      【7 空】碰撞与同哈希程序
│   ├── task4_defense.py        【4 空】HMAC 防御
│   ├── verify.py               交付物核对
│   └── run_all.py              一键跑全链
├── data/
    ├── collision_a.bin         128 字节碰撞块 A
    ├── collision_b.bin         128 字节碰撞块 B
    └── sample_url.txt          一条合法的银行 URL
```

### 2.3 数据说明

| 文件 | 大小 | 说明 |
|---|---|---|
| `collision_a.bin` | 128 字节 | 王小云 2004 碰撞块 A |
| `collision_b.bin` | 128 字节 | 和 A 只差 6 个比特 |
| `sample_url.txt` | 约 120 字节 | 合法 URL，token 是攻击的输入 |

---

## 3. 场景

你在一家银行做安全测试。银行有一个内部 API：

```
GET /api?token=...&user=alice&command1=ListSquirrels&command2=NoOp
```

服务端用 `MD5(password || 消息)` 校验请求。密码 8 个字符，只有服务端知道。 你手上有一条合法 URL，是 alice 的普通查询。

`api_common.py` 是本地模拟的服务端。做任务二时把它当黑盒：只用 token、消息和密码长度，不要读源码里的 `SECRET`。

你的目标：**不改密码，不加 command1、command2，追加一条**

```
&command3=UnlockAllSafes
```

让服务端返回 200，并且执行这条命令。

第二条任务线是软件分发。银行准备发布一个 Python 小工具，下载页只显示 MD5。 你要证明：**两个 MD5 相同的文件，可以一个安全、一个有害。**

---

## 4. 填空怎么填

### 4.1 填空长什么样

骨架是完整可运行的，只在关键位置留了空：

```
head = FILL[1]          # 表达式位：把 FILL[1] 换成你的表达式
```

```
# TODO(4): 把十六进制摘要还原成 16 字节状态。
state = FILL[4]          # 表达式位


# TODO(7): 把 suffix 喂进去。
FILL[7]                  # 语句位：删掉 FILL[7]，原位写语句
```

每个 TODO 都有编号，和本文 §5、附录 E 对应。

没填时，运行到那一行会抛出：

```
FILL[1] not filled yet -- task1_md5.py 第 50 行:
    | head = FILL[1]
把 FILL[1] 这几个字删掉，在原位写你的代码。
```

## 4.2 三条注意事项

**事项一：把 `FILL[n]` 这几个字符删掉，在原位写代码。不要给它赋值。**

```
FILL[4] = bytes.fromhex(digest_hex)    # ✗ 错，会抛 TypeError
state = bytes.fromhex(digest_hex)      # ✓ 对
```

**事项二：一处填空 = 一个完整的表达式或一条完整的语句。**

```
pad = FILL[4]                    # ✓
h.update(FILL[7])                # ✗ 不要嵌进更大的表达式
```

`h.update(FILL[7])` 这种写法会把整行替换掉，导致 `h.update(` 丢失。 本包的填空都单独占一行，照骨架写即可。

**事项三：填空只出现在函数体里。**

文件可以随时 import、可以 `--help`，只有执行到那一行才会失败。

## 4.3 怎么知道还剩几个空

```
python run_all.py --progress
```

输出：

```
task1_md5.py            ▒▒▒▒▒▒   6 / 6
task2_extend.py         ▒▒▒▒▒▒▒▒  8 / 8
task3_collision.py      ▒▒▒▒▒▒▒  7 / 7
task4_defense.py        ▒▒▒▒   4 / 4

合计 25 / 25（25 处待填）
```

前面的数字是**还没填**的空数；`6 / 6` 表示 6 处都还没填。填完一处，数字就减一。

不要用 `grep -c 'FILL\['` 数空：注释里也会出现 `FILL[n]`，会数多。 以 `run_all.py --progress` 为准。

### 4.4 「还没填」和「填错了」是两回事

| 情况 | 退出码 | 表现 |
|---|---|---|
| 空还没填 | **3** | `FILL[n] not filled yet` + 文件名 + 行号 |
| 填错了 | **1** | 自检里的具体失败项 |
| 全对 | 0 | 全部通过 |

退出码 3 是正常状态。

---

## 5. 四个任务

每个任务固定五段：**目的** → **命令** → **实测输出** → **要求** → **关键结论。**

### 5.1 任务一　填充与续写【6 处填空】

**目的**

自己写出 MD5 填充，并用旧摘要当初始状态续算一段 suffix。 这是任务二的内核。

**命令**

```
cd code
python task1_md5.py
```

**实测输出**

```
一、填充长什么样
  消息   0 字节 → 填充  64 字节，补后 0 mod 64  开头 8000000000000000
  消息   3 字节 → 填充  61 字节，补后 0 mod 64  开头 8000000000000000
  消息  55 字节 → 填充   9 字节，补后 0 mod 64  开头 8000000000000000
  消息  56 字节 → 填充  72 字节，补后 0 mod 64  开头 8000000000000000
  消息  64 字节 → 填充  64 字节，补后 0 mod 64  开头 8000000000000000

二、用旧摘要续写一段 suffix
  original = b'Use HMAC, not hashes'
  H(original) = [···]
  填充 = [···]

  从旧状态续写 suffix：[···]
  直接哈希整条长消息：[···]
  两者相同：True
```

**要求**

1. 填 `FILL[1]` ~ `FILL[3]`，完成 `padding()`；
2. 填 `FILL[4]` ~ `FILL[6]`，完成 `extend_digest()`；
3. 记录 55、56 两个边界长度的填充字节数，以及示例消息的原消息长度、填充长度、`H(original)`、续写摘要、直接摘要（报告任务一第 1、2 题）。

**关键结论**

1. 填充由三部分组成：`0x80`、若干 `0x00`、8 字节小端序比特长度。
2. 55 字节消息填充 9 字节；56 字节消息填充 72 字节。多出来的是**一整块**。
3. 续写只需要 `H(original)` 和 `len(original)`，不需要 `original` 的内容。
4. `FILL[6]` 必须为 `True`。对不上时，先查零的个数，再查 `count` 有没有乘 8。

---

## 5.2 任务二　长度扩展攻击【8 处填空】【★ 主交付物】

**目的**

在不知道密码的情况下，伪造一条带 `command3=UnlockAllSafes` 的银行请求。

**命令**

```
python task2_extend.py
python api_common.py (Get-Content ../data/forged_url.txt)
```

> cmd 里改成 `set /p URL=<..\data\forged_url.txt`，再 `python api_common.py "%URL%"`。

**实测输出**

```
原始 URL 的消息部分（47 字节）：
    user=alice&command1=ListSquirrels&command2=NoOp
  token：[···]
  已知密码长度：8

  原始请求：
    status = 200
    unlock = False

  伪造请求：
    status = 200
    unlock = True
    command3 = UnlockAllSafes

  填充长度：[···] 字节
  新消息长度：[···] 字节

  ★ 成功。服务端认为这条请求是 alice 发的。
```

**要求**

1. 填 `FILL[1]` ~ `FILL[3]`，拆出 token、原始查询串、消息字节；
2. 填 `FILL[4]` ~ `FILL[8]`，构造填充、续算 token、拼回 URL；
3. `data/forged_url.txt` 里的 URL 必须让服务端返回 200 且 `unlock=True`；
4. 记录原始/伪造请求的 status、`unlock` 和 token 前 8 位，以及原消息、填充、新消息长度（报告任务二第 1 题）。

**关键结论**

1. 填充长度按 `len(password) + len(msg)` 算，不是只按 `len(msg)`。
2. `count` 是已经处理过的**比特数**，要乘 8，并且包含填充。
3. `suffix` 要同时出现在两处：拼进 `new_msg`，喂给 `h.update()`。
4. URL 编码用 `quote_from_bytes(new_msg, safe="")`，让 `&`、`=`、`%` 都按字节编码。
5. 服务端没有泄露密码。它只是相信了「token 对得上 = 消息没被改过」这个错误前提。

**常见错误**

| 现象 | 原因 |
| --- | --- |
| `status=403` | 填充长度没加密码长度，或 `count` 没乘 8 |
| `status=200` 但 `unlock=False` | suffix 没拼进 `new_msg`，或没喂给 `h.update()` |
| URL 解析后消息长度不对 | `quote_from_bytes` 的 `safe` 不是空字符串 |
| `unlock` 为 `True` 但 token 没变 | 用了旧 token 直接拼，没有续算 |

## 5.3 任务三　碰撞与同哈希程序【7 处填空】

### 目的

验证一对 MD5 碰撞块的性质，并做出两个「MD5 相同、行为不同」的可执行文件。

### 命令

```
python task3_collision.py
python ../data/good
python ../data/evil
```

### 实测输出

```
碰撞块 A：128 字节
碰撞块 B：128 字节

两个碰撞块：
  MD5     A = 79054025255fb1a26e4bc422aef54eb4
  MD5     B = 79054025255fb1a26e4bc422aef54eb4    相同
  SHA-256 A = 8d12236e5c4ed9f4e790db4d868fd5c3…
  SHA-256 B = b9fef2a8fc93b05e7701e97196fda6c4…    不同

逐字节比较：6 个字节不同，合计 6 个比特。
  偏移    A      B      异或    变了几个比特
    19   0x87   0x07   0x80      1
    45   0x87   0x07   0x80      1
    ...

碰撞块换个位置：
  加共同前缀（34 字节）：MD5 还相同？ False
  加共同后缀（22 字节）：MD5 还相同？ True

两个程序：
  good  381 字节  MD5 [···]
  evil  381 字节  MD5 [···]
  MD5 相同：True
  SHA-256 相同：False
  good 输出：I mean no harm.
  evil 输出：You are doomed!
```

### 要求

1. 填 `FILL[1]`，逐字节找出 A、B 的差异；
2. 填 `FILL[2]`、`FILL[3]`，把 `PAYLOAD_CODE` 打成 zipapp；
3. 填 `FILL[4]` ~ `FILL[6]`，把碰撞块接在 zipapp 前面并运行；
4. 填 `FILL[7]`，验证两个文件 MD5 相同；

5. 记录碰撞块的 MD5、SHA-256 是否相同、差异字节/比特、共同前缀/后缀结论，以及 good/evil 的 MD5 和输出（报告任务三第 1 题）。

**关键结论**

1. 两个 128 字节文件只有 6 个比特不同，MD5 完全相同。
2. 碰撞是「起始状态 + 消息」的性质：共同前缀会破坏它，共同后缀不会。
3. 两个程序共用同一段代码，差别只在文件开头的碰撞块里。
4. 只校验 MD5 的下载页会把 evil 当成 good。
5. MD5 已经不适合做文件完整性校验；需要抗碰撞时用 SHA-256 或 SHA-3。

---

## 5.4 任务四　HMAC 防御【4 处填空】

**目的**

把 SHA-256(secret || m) 和 HMAC-SHA-256(secret, m) 放在一起对比， 理解为什么前者为什么防御失败。

**命令**

```
python task4_defense.py
```

**实测输出**

```
同一条消息，两种写法：
  SHA-256(secret || m) = [···]
  HMAC-SHA-256         = [···]

长度扩展演示（用 MD5 做同构演示）：
  H(secret || m)            = [···]
  续写后                    = [···]
  直接哈希整条长消息        = [···]
  相同：True

HMAC 校验：
  正确 tag 通过：True
  错误 tag 通过：False

★ 结论：
  1. H(secret || m) 在知道长度时可以被续写；
  2. HMAC 不是「把 secret 拼进去再哈希」，它把 secret 处理成
     两个密钥块，外层再哈希一次，所以不能从 tag 续写；
  3. 需要 MAC 时，用 hmac 模块，不要自己拼字符串。
```

**要求**

1. 填 `FILL[1]`，写出 `SHA-256(secret || m)`；
2. 填 `FILL[2]`，用 `hmac.new` 写 HMAC；
3. 填 `FILL[3]`，用 `hmac.compare_digest` 校验；
4. 填 `FILL[4]`，让正确 tag 通过；
5. 记录两种写法的值、长度扩展演示、正确/错误 tag 的校验结果（报告任务四第 1 题）。

**关键结论**

1. `H(secret || m)` 可被长度扩展；HMAC 不能。
2. HMAC 的内外层都用了密钥，攻击者无法把 tag 当作中间状态。
3. 比较 tag 必须用恒定时间比较，避免逐字节比较泄露信息。
4. 密码学代码优先用标准库，不要自己发明 MAC 构造。

---

# 6. 报告要求

## 6.1 提交清单

| # | 文件 | 说明 |
|---|---|---|
| 1 | `data/forged_url.txt` | 主交付物，服务端返回 200 且执行 UnlockAllSafes |
| 2 | `data/good`、`data/evil` | 两个可执行文件，MD5 相同，行为不同 |
| 3 | `data/task1_padding.json` | 任务一的填充与续写数据 |
| 4 | `data/defense.json` | 任务四的对照结果 |
| 5 | `code/` 里填好的四个 `task*.py` | 完整代码 |
| 6 | 实验报告（Word） | 按模板作答 6 个问题 |

---

## 环境和运行

| 现象 | 原因 | 怎么办 |
|---|---|---|
| `ModuleNotFoundError: No module named 'fill'` | 用了 `python -I` 或 `-E` | 去掉参数 |
| 终端中文乱码 | 终端不是 UTF-8 | `python -X utf8 xxx.py` |
| `python` 是 Microsoft Store 转发器 | Windows 应用别名 | 用 `py -3 xxx.py` |
| `data/good` 运行时报 `SyntaxError` | 用文本编辑器改过二进制文件 | 重新运行 `python task3_collision.py` |